



Lab Assessment Report

Only for Course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	15
Problem Analysis	03					
Solution Design	02					
Code Development	06					
Accuracy	04					
Total obtained mark						
Comments						

Semester: Spring 2026

Student Name: MD.Rifat Mahmud Emon

Student ID: 242-35-527

Batch: 43

Section: L1

Course Code: SE215 Course Name: Algorithm Design & Analysis

Course Teacher Name: Syed Shams Islam

Designation: Lecturer

Submission Date: 07-04-2026

Algorithm Design & Analysis

Lab Report

Rifat Mahmud Emon

ID: 242-35-527

Section: 43L1

Problem-1: Median via Selection Sort Invariant

Code:

```
#include <stdio.h>

int find_median(int *arr, int size) {
    if (size == 0) return -1;
    int i, j, min_idx, temp;
    int mid = size / 2;
    for (i = 0; i <= mid; i++) {
        min_idx = i;
        for (j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
    return arr[mid];
}
```

```

int main() {

    int arr[] = {7, 2, 10, 9, 1};

    int size = 5;

    int median = find_median(arr, size);

    printf("%d\n", median);

    return 0;

}

```

The screenshot shows a C++ IDE with a file named 'main.c'. The code defines a function 'find_median' that uses a selection sort algorithm to find the median of an array. The 'main' function calls 'find_median' with the array {7, 2, 10, 9, 1} and size 5. The output window shows the result '7' and a message '=== Code Execution Successful ==='.

```

1 #include <stdio.h>
2 int find_median(int *arr, int size) { if (size == 0) return -1;
3 int i, j, min_idx, temp; int mid = size / 2;
4 for (i = 0; i <= mid; i++) { min_idx = i;
5 for (j = i + 1; j < size; j++) { if (arr[j] < arr[min_idx]) {
6 min_idx = j;
7 }
8 }
9 temp = arr[i];
10 arr[i] = arr[min_idx]; arr[min_idx] = temp;
11 }
12 return arr[mid];
13 }
14
15 int main() {
16 int arr[] = {7, 2, 10, 9, 1};
17 int size = 5;
18 int median = find_median(arr, size); printf("%d\n", median);
19 return 0;
20 }

```

Output: 7

=== Code Execution Successful ===

Problem-2: K-th Largest via Bubble Sort Invariant

Code:

```

#include <stdio.h>

int find_kth_largest(int *arr, int size, int k) {

    int i, j, temp;

    for (i = 0; i < k; i++) {

        for (j = 0; j < size - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

}

```

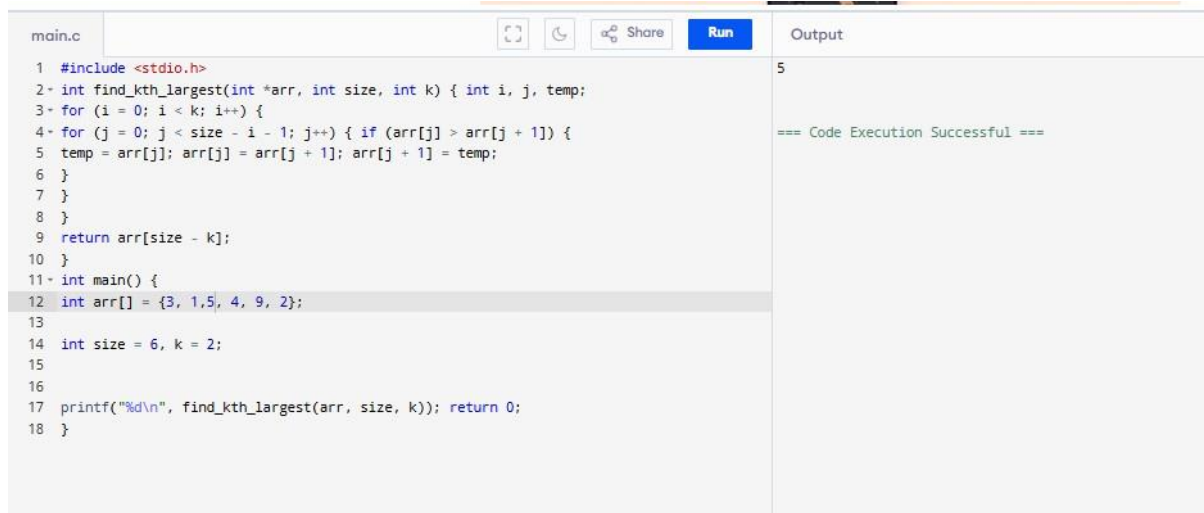
```
    return arr[size - k];  
}  
  
int main() {  
    int arr[] = {3, 1, 7, 4, 9, 2};
```

```
int size = 6, k = 2;
```

```
printf("%d\n", find_kth_largest(arr, size, k));
```

```
return 0;
```

```
}
```



The screenshot shows a C++ IDE with a file named 'main.c'. The code defines a function 'find_kth_largest' that uses a selection sort-like approach to find the kth largest element in an array. The array is {3, 1, 5, 4, 9, 2}, size is 6, and k is 2. The output of the program is 5, indicating the 2nd largest element. The IDE interface includes a 'Run' button and a 'Share' link. The output window shows '5' and '=== Code Execution Successful ==='.

```
1 #include <stdio.h>
2 int find_kth_largest(int *arr, int size, int k) { int i, j, temp;
3 for (i = 0; i < k; i++) {
4 for (j = 0; j < size - i - 1; j++) { if (arr[j] > arr[j + 1]) {
5 temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
6 }
7 }
8 }
9 return arr[size - k];
10 }
11 int main() {
12 int arr[] = {3, 1, 5, 4, 9, 2};
13
14 int size = 6, k = 2;
15
16
17 printf("%d\n", find_kth_largest(arr, size, k)); return 0;
18 }
```

Output: 5

=== Code Execution Successful ===

Problem-3: Merge Two Arrays via Insertion Sort Logic

Code:

```
#include <stdio.h>
```

```
void merge_sorted(int *a, int n, int *b, int m, int *out) {
```

```
    int i, j, key;
```

```
    for (i = 0; i < n + m; i++) {
```

```
        if (i < n)
```

```
            key = a[i];
```

```
        else
```

```
            key = b[i - n];
```

```
        j = i - 1;
```

```
        while (j >= 0 && out[j] > key) {
```

```
            out[j + 1] = out[j];
```

```
            j--;
```

```
        }
```

```
        out[j + 1] = key;
    }
}

int main() {
    int a[] = {5, 1, 9};
```

```

int b[] = {3, 7, 2, 6};

int n = 3, m = 4;

int out[7];

merge_sorted(a, n, b, m, out);

for (int i = 0; i < n + m; i++) {

    printf("%d ", out[i]);

}

return 0;

}

```

main.c	Output
<pre> 1 #include <stdio.h> 2 void merge_sorted(int *a, int n, int *b, int m, int *out) { int i, j, key; 3 for (i = 0; i < n + m; i++) { if (i < n) 4 key = a[i]; else 5 key = b[i - n]; j = i - 1; 6 while (j >= 0 && out[j] > key) { out[j + 1] = out[j]; 7 j--; 8 } 9 out[j + 1] = key; 10 } 11 } 12 int main() { 13 int a[] = {5, 10, 9}; 14 15 int b[] = {13, 7, 12, 6}; 16 int n = 3, m = 4; int out[7]; 17 merge_sorted(a, n, b, m, out); for (int i = 0; i < n + m; i++) { 18 printf("%d ", out[i]); 19 } 20 return 0; 21 } </pre>	<pre> 5 6 7 9 10 12 13 === Code Execution Successful === </pre>

Problem-4.1: Factorial

Code:

```

#include <stdio.h>

long long factorial(int n) {

    long long result = 1;

    int i;

    for (i = 1; i <= n; i++) {

        result = result * i;

    }

}

```

```
    return result;
}

int main() {

    int n = 5;

    printf("%lld\n", factorial(n));

    return 0;
}
```

main.c	Run	Output
<pre>1 #include <stdio.h> 2 long long factorial(int n) { long long result = 1; 3 int i; 4 for (i = 1; i <= n; i++) { result = result * i; 5 } 6 return result; 7 } 8 int main() { int n = 7; 9 printf("%lld\n", factorial(n)); return 0; 10 } 11</pre>		<pre>5040 === Code Execution Successful ===</pre>

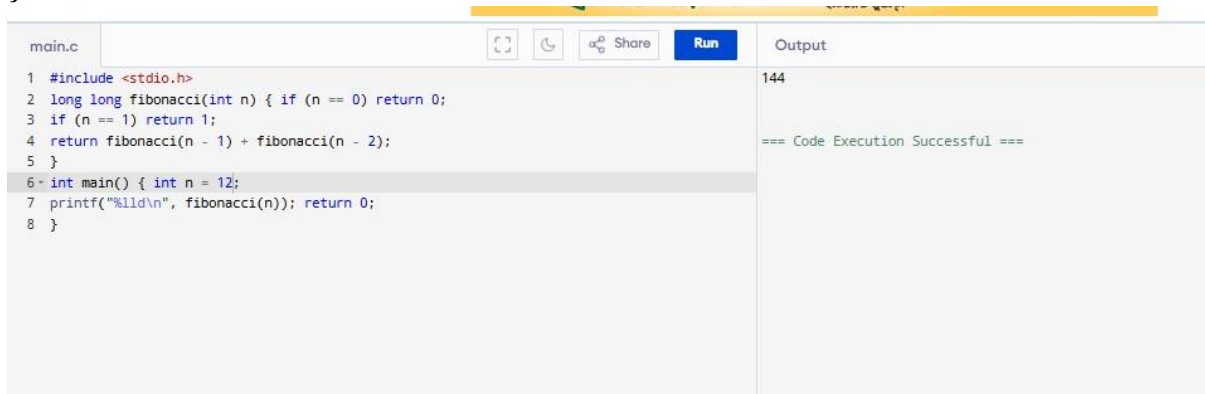
Problem-4.2: Fibonacci

Code:

```
#include <stdio.h>

long long fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n = 10;
    printf("%lld\n", fibonacci(n));
    return 0;
}
```

A screenshot of a C code editor interface. The editor has a tab labeled 'main.c'. The code is as follows:

```
1 #include <stdio.h>
2 long long fibonacci(int n) { if (n == 0) return 0;
3   if (n == 1) return 1;
4   return fibonacci(n - 1) + fibonacci(n - 2);
5 }
6 int main() { int n = 12;
7   printf("%lld\n", fibonacci(n)); return 0;
8 }
```

The editor has buttons for 'Share', 'Run', and 'Output'. The 'Output' panel on the right shows the result '144' and the message '=== Code Execution Successful ==='.

Problem-4.3: Count Digits

Code:

```
#include <stdio.h>

int count_digits(int n) {
    if (n == 0) return 1;
    if (n < 10) return 1;
    return 1 + count_digits(n / 10);
}

int main() {
```

```
int n = 98765;

printf("%d\n", count_digits(n));

return 0;
}
```

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int count_digits(int n) { if (n == 0) return 1; 3 if (n < 10) return 1; 4 return 1 + count_digits(n / 10); 5 } 6 int main() { 7 int n = 98765; 8 printf("%d\n", count_digits(n)); return 0; 9 } 10</pre>		<pre>5 === Code Execution Successful ===</pre>

Problem-4.4: Power Function

Code:

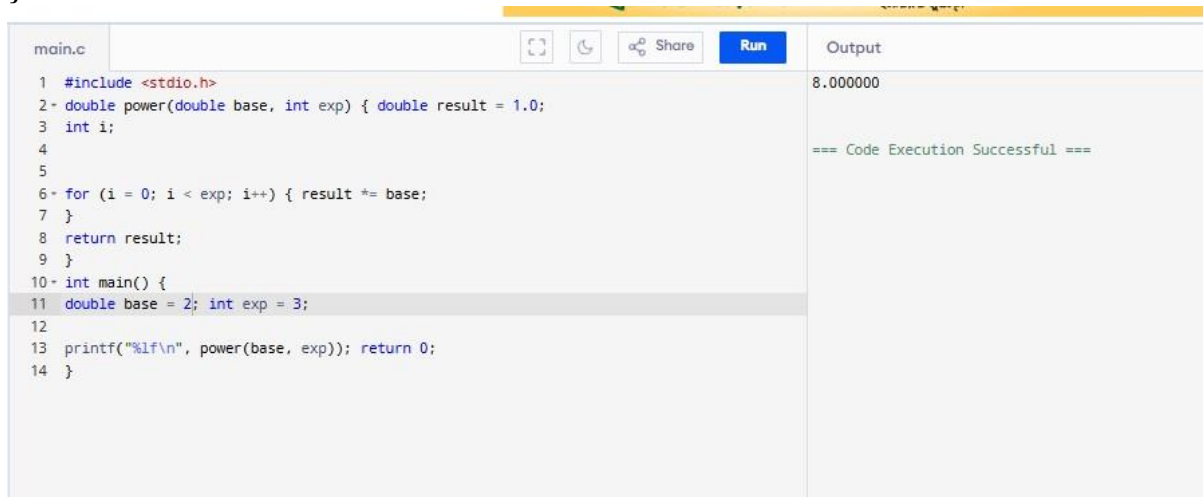
```
#include <stdio.h>

double power(double base, int exp) {
    double result = 1.0;
    int i;

    for (i = 0; i < exp; i++) {
        result *= base;
    }
    return result;
}

int main() {
    double base = 2.5;
    int exp = 3;

    printf("%lf\n", power(base, exp));
    return 0;
}
```



The screenshot shows a C code editor with the following code:

```
main.c
1 #include <stdio.h>
2 double power(double base, int exp) { double result = 1.0;
3 int i;
4
5
6 for (i = 0; i < exp; i++) { result *= base;
7 }
8 return result;
9 }
10 int main() {
11 double base = 2; int exp = 3;
12
13 printf("%lf\n", power(base, exp)); return 0;
14 }
```

The output is displayed as 8.000000. The code execution is successful.

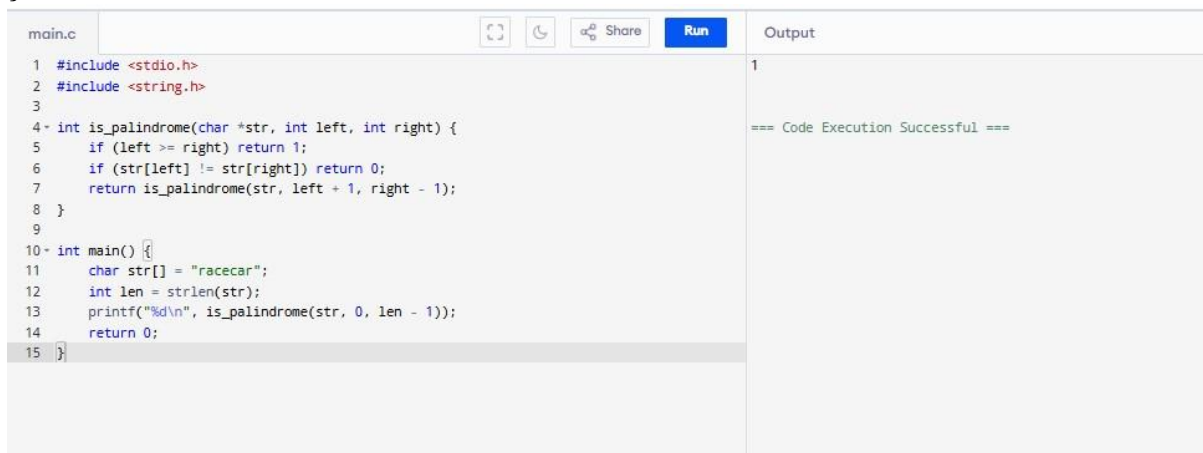
Problem-4.5: Palindrome Check

Code:

```
#include <stdio.h>
#include <string.h>

int is_palindrome(char *str, int left, int right) {
    if (left >= right) return 1;
    if (str[left] != str[right]) return 0;
    return is_palindrome(str, left + 1, right - 1);
}

int main() {
    char str[] = "racecar";
    int len = strlen(str);
    printf("%d\n", is_palindrome(str, 0, len - 1));
    return 0;
}
```



```
main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 int is_palindrome(char *str, int left, int right) {
5     if (left >= right) return 1;
6     if (str[left] != str[right]) return 0;
7     return is_palindrome(str, left + 1, right - 1);
8 }
9
10 int main() {
11     char str[] = "racecar";
12     int len = strlen(str);
13     printf("%d\n", is_palindrome(str, 0, len - 1));
14     return 0;
15 }
```

Output

1

=== Code Execution Successful ===

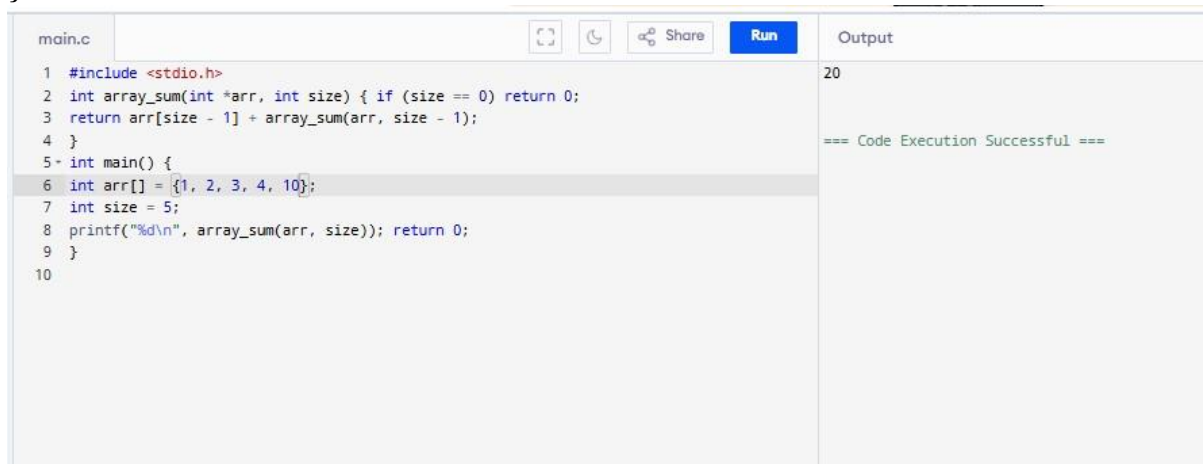
Problem-4.6: Sum of Array

Code:

```
#include <stdio.h>

int array_sum(int *arr, int size) {
    if (size == 0) return 0;
    return arr[size - 1] + array_sum(arr, size - 1);
}
```

```
}  
  
int main() {  
    int arr[] = {1, 2, 3, 4, 5};  
  
    int size = 5;  
  
    printf("%d\n", array_sum(arr, size));  
  
    return 0;  
}
```



The screenshot shows a code editor with a file named 'main.c'. The code defines a recursive function 'array_sum' and a 'main' function. The 'array_sum' function takes an array and its size, and returns the sum of its elements. The 'main' function initializes an array with values {1, 2, 3, 4, 10}, sets the size to 5, and prints the result of 'array_sum(arr, size)'. The output of the program is '20', and a message '=== Code Execution Successful ===' is displayed.

```
main.c  
1 #include <stdio.h>  
2 int array_sum(int *arr, int size) { if (size == 0) return 0;  
3 return arr[size - 1] + array_sum(arr, size - 1);  
4 }  
5 int main() {  
6 int arr[] = {1, 2, 3, 4, 10};  
7 int size = 5;  
8 printf("%d\n", array_sum(arr, size)); return 0;  
9 }  
10
```

Output

20

=== Code Execution Successful ===

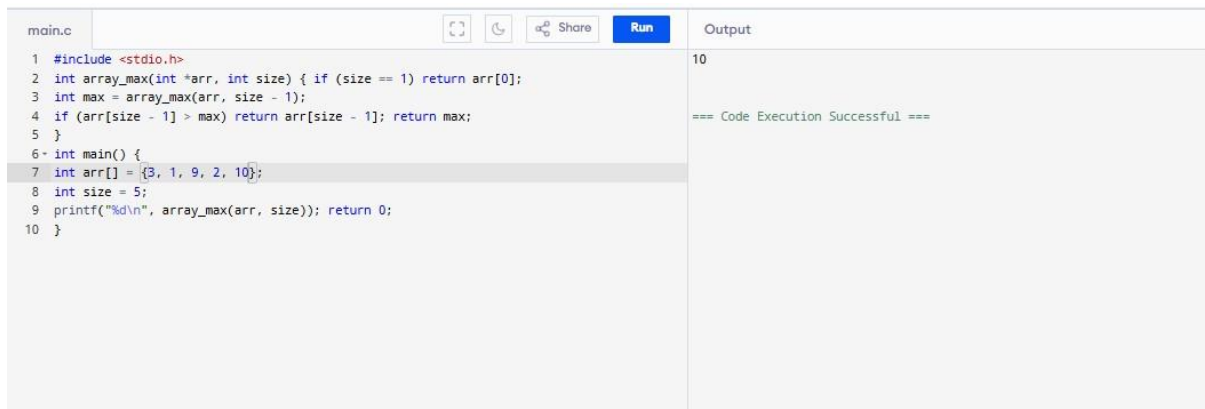
Problem-4.7: Maximum of Array

Code:

```
#include <stdio.h>

int array_max(int *arr, int size) {
    if (size == 1) return arr[0];
    int max = array_max(arr, size - 1);
    if (arr[size - 1] > max) return arr[size - 1];
    return max;
}

int main() {
    int arr[] = {3, 1, 9, 2, 7};
    int size = 5;
    printf("%d\n", array_max(arr, size));
    return 0;
}
```



```
main.c  [Icons]  Run  Output
1 #include <stdio.h>
2 int array_max(int *arr, int size) { if (size == 1) return arr[0];
3 int max = array_max(arr, size - 1);
4 if (arr[size - 1] > max) return arr[size - 1]; return max;
5 }
6 int main() {
7 int arr[] = {3, 1, 9, 2, 10};
8 int size = 5;
9 printf("%d\n", array_max(arr, size)); return 0;
10 }

Output
10

=== Code Execution Successful ===
```

Problem-4.8: Binary Search

Code:

```
#include <stdio.h>

int binary_search(int *arr, int left, int right, int target)
{
    if (left > right)
```

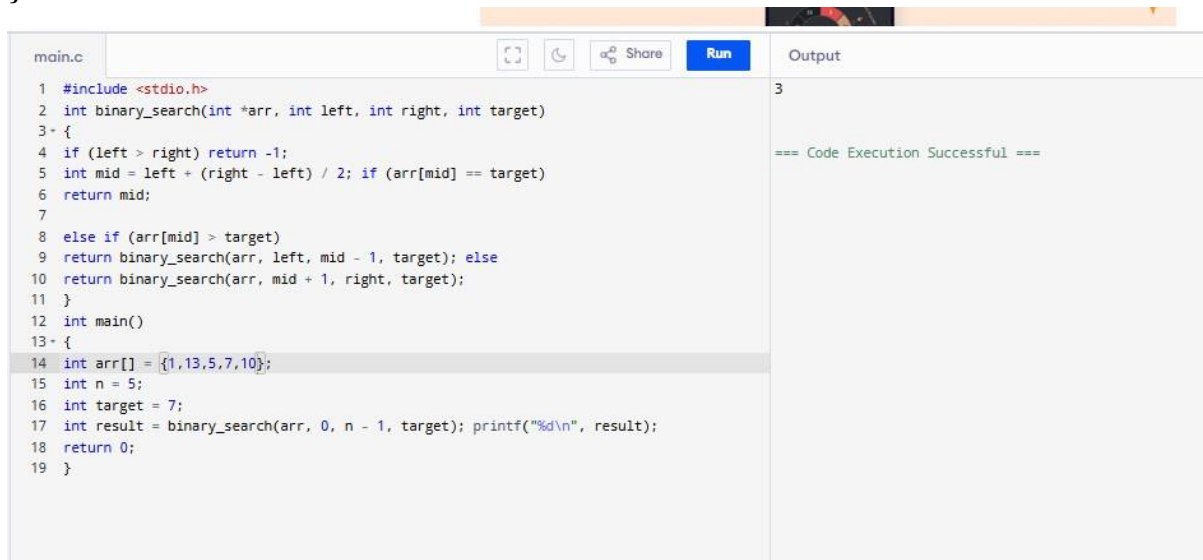
```
    return -1;
int mid = left + (right - left) / 2;
if (arr[mid] == target)
    return mid;
```

```

else if (arr[mid] > target)
    return binary_search(arr, left, mid - 1, target);
else
    return binary_search(arr, mid + 1, right, target);
}

int main()
{
    int arr[] = {1,3,5,7,9};
    int n = 5;
    int target = 7;
    int result = binary_search(arr, 0, n - 1, target);
    printf("%d\n", result);
    return 0;
}

```



The screenshot shows a C code editor with a file named 'main.c'. The code implements a binary search function and a main function. The array is {1, 13, 5, 7, 10} and the target is 7. The output is 3, indicating the index of the target. The code execution was successful.

```

main.c
1 #include <stdio.h>
2 int binary_search(int *arr, int left, int right, int target)
3 {
4     if (left > right) return -1;
5     int mid = left + (right - left) / 2; if (arr[mid] == target)
6     return mid;
7
8     else if (arr[mid] > target)
9     return binary_search(arr, left, mid - 1, target); else
10    return binary_search(arr, mid + 1, right, target);
11 }
12 int main()
13 {
14     int arr[] = {1,13,5,7,10};
15     int n = 5;
16     int target = 7;
17     int result = binary_search(arr, 0, n - 1, target); printf("%d\n", result);
18     return 0;
19 }

```

Output: 3

=== Code Execution Successful ===

Problem-5: Merge sort

Code:

```

#include <stdio.h>

void merge(int *arr, int left, int mid, int right)
{

```



```
int n1 = mid - left + 1;  
int n2 = right - mid;  
int L[n1], R[n2];  
for (int i = 0; i < n1; i++)  
    L[i] = arr[left + i];  
for (int i = 0; i < n2; i++)
```

```

        R[i] = arr[mid + 1 + i];
int i = 0, j = 0, k = left;
while (i < n1 && j < n2)
{
    if (L[i] <= R[j])
        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}
while (i < n1)
    arr[k++] = L[i++];
while (j < n2)
    arr[k++] = R[j++];
}
void merge_sort(int *arr, int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main()
{

```

```

int arr[] = {38, 27, 43, 3, 9, 82, 10};

int n = 7;

merge_sort(arr, 0, n - 1);

for (int i = 0; i < n; i++)

    printf("%d ", arr[i]);

return 0;

}

```

main.c	Output
<pre> 1 #include <stdio.h> 2 void merge(int *arr, int left, int mid, int right) 3 { 4 int n1 = mid - left + 1; int n2 = right - mid; int L[n1], R[n2]; 5 for (int i = 0; i < n1; i++) L[i] = arr[left + i]; 6 for (int i = 0; i < n2; i++) 7 8 R[i] = arr[mid + 1 + i]; int i = 0, j = 0, k = left; while (i < n1 && j < n2) 9 { 10 if (L[i] <= R[j]) 11 arr[k++] = L[i++]; else 12 arr[k++] = R[j++]; 13 } 14 while (i < n1) arr[k++] = L[i++]; 15 while (j < n2) arr[k++] = R[j++]; 16 } 17 void merge_sort(int *arr, int left, int right) 18 { 19 if (left < right) 20 { 21 int mid = left + (right - left) / 2; merge_sort(arr, left, mid); merge_sort(arr, mid + 22 1, right); merge(arr, left, mid, right); 23 } 24 } 25 int main() 26 { 27 int arr[] = {18, 17, 43, 3, 19, 8, 10}; 28 int n = 7; merge_sort(arr, 0, n - 1); for (int i = 0; i < n; i++) 29 printf("%d ", arr[i]); return 0; 30 } </pre>	<pre> 3 8 10 17 18 19 43 === Code Execution Successful === </pre>

Proben-6: Quick sort

Code:

```

#include <stdio.h>

int swap_count = 0;

void swap(int *a, int *b)

{

    int t = *a;

    *a = *b;

    *b = t;

    swap_count++;

}

int partition(int *arr, int low, int high)

```

```
{  
    int pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++)  
    {  
        if (arr[j] < pivot)  
        {
```

```

        i++;

        swap(&arr[i], &arr[j]);
    }
}

swap(&arr[i + 1], &arr[high]);

return i + 1;
}

void quick_sort(int *arr, int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

int main()
{
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = 6;
    quick_sort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nSwaps: %d\n", swap_count);
    return 0;
}

```

main.c

Share

Run

```
10 //
11
12 int partition(int *arr, int low, int high) {
13     int pivot = arr[high];
14     int i = low - 1;
15     for (int j = low; j < high; j++) {
16         if (arr[j] < pivot) {
17             i++;
18             swap(&arr[i], &arr[j]);
19         }
20     }
21     swap(&arr[i + 1], &arr[high]);
22     return i + 1;
23 }
24
25 void quick_sort(int *arr, int low, int high) {
26     if (low < high) {
27         int pi = partition(arr, low, high);
28         quick_sort(arr, low, pi - 1);
29         quick_sort(arr, pi + 1, high);
30     }
31 }
32
33 int main() {
34     int arr[] = {10, 7, 8, 9, 1, 5};
35     int n = 6;
36     quick_sort(arr, 0, n - 1);
37     for (int i = 0; i < n; i++)
38         printf("%d ", arr[i]);
39     printf("\nSwaps: %d\n", swap_count);
40     return 0;
41 }
```

Output

1 5 7 8 9 10
Swaps: 5

=== Code Execution Successful ===

Problem-7: Fractional Knapsack

Code:

```
#include <stdio.h>

#include <stdlib.h>

typedef struct {
    int weight;
    int value;
} Item;

int compare(const void *a, const void *b)
{
    double r1 = (double)((Item *)b)->value / ((Item *)b)->weight;
    double r2 = (double)((Item *)a)->value / ((Item *)a)->weight;
    if (r1 > r2) return 1;
    else if (r1 < r2) return -1;
    return 0;
}

double fractional_knapsack(Item *items, int n, int capacity)
{
    qsort(items, n, sizeof(Item), compare);
    double total = 0.0;
    for (int i = 0; i < n; i++)
    {
        if (capacity >= items[i].weight)
        {
            capacity -= items[i].weight;
            total += items[i].value;
        }
    }
}
```

```

    }
    else
    {
        total += (double)items[i].value * capacity / items[i].weight;
        break;
    }
}

return total;
}

int main()
{
    Item items[] = {{10,60},{20,100},{30,120}};

    int n = 3;

    int capacity = 50;

    double result = fractional_knapsack(items, n, capacity);

    printf("%.2lf\n", result);

    return 0;
}

```

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int weight;
6     int value;
7 } Item;
8
9 int compare(const void *a, const void *b) {
10     double r1 = (double)((Item *)a)->value / ((Item *)a)->weight;
11     double r2 = (double)((Item *)b)->value / ((Item *)b)->weight;
12     if (r1 > r2) return -1;
13     else if (r1 < r2) return 1;
14     return 0;
15 }
16
17 double fractional_knapsack(Item *items, int n, int capacity) {
18     qsort(items, n, sizeof(Item), compare);
19     double total = 0.0;
20     for (int i = 0; i < n; i++) {
21         if (capacity >= items[i].weight) {
22             capacity -= items[i].weight;
23             total += items[i].value;
24         } else {
25             total += (double)items[i].value * capacity / items[i].weight;
26             break;
27         }
28     }
29     return total;
30 }
31
32 int main() {
33     Item items[] = {{10,60},{20,100},{30,120}};
34     int n = 3;
35     int capacity = 50;
36     double result = fractional_knapsack(items, n, capacity);
37     printf("%.2lf\n", result);
38     return 0;
39 }

```

Output

```

240.00
=== Code Execution Successful ===

```


Problem-8: Minimum Spanning Tree

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

typedef struct {

    int u, v, w;
```

```

} Edge;

int find(int parent[], int i)
{
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

void unite(int parent[], int x, int y)
{
    parent[find(parent, x)] = find(parent, y);
}

void prims(int g[5][5], int V)
{
    int key[5], parent[5], used[5];
    for(int i=0;i<V;i++) key[i]=INT_MAX, used[i]=0;
    key[0]=0; parent[0]=-1;
    for(int i=0;i<V-1;i++)
    {
        int min=INT_MAX,u;
        for(int j=0;j<V;j++)
            if(!used[j] && key[j]<min) min=key[j],u=j;
        used[u]=1;
        for(int v=0;v<V;v++)
            if(g[u][v] && !used[v] && g[u][v]<key[v])
                key[v]=g[u][v], parent[v]=u;
    }

    int total=0;

```

```

printf("Prim:\n");
for(int i=1;i<V;i++)
{
    printf("%d-%d %d\n", parent[i], i, g[i][parent[i]]);
    total+=g[i][parent[i]];
}
printf("Total %d\n\n", total);
}

int cmp(const void *a,const void *b)
{
    return ((Edge*)a)->w - ((Edge*)b)->w;
}

void kruskal(Edge e[], int E, int V)
{
    qsort(e,E,sizeof(Edge),cmp);
    int parent[5];
    for(int i=0;i<V;i++) parent[i]=i;
    int total=0,c=0;
    printf("Kruskal:\n");
    for(int i=0;i<E && c<V-1;i++)
    {
        int u=e[i].u, v=e[i].v;
        if(find(parent,u)!=find(parent,v))
        {
            printf("%d-%d %d\n",u,v,e[i].w);
            total+=e[i].w;

```

```

        unite(parent,u,v);
        c++;
    }
}
printf("Total %d\n", total);
}
int main()
{
    int g[5][5]={
        {0,2,0,6,0},
        {2,0,3,8,5},
        {0,3,0,0,7},
        {6,8,0,0,9},
        {0,5,7,9,0}
    };
    Edge e[]={
        {0,1,2},{0,3,6},{1,2,3},
        {1,3,8},{1,4,5},{2,4,7},{3,4,9}
    };
    prims(g,5);
    kruskal(e,7,5);
    return 0;
}

```

main.c

Share

Run

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <limits.h>
4
5 typedef struct {
6     int u, v, w;
7 } Edge;
8
9 int find(int parent[], int i) {
10     if (parent[i] == i) return i;
11     return parent[i] = find(parent, parent[i]);
12 }
13
14 void unite(int parent[], int x, int y) {
15     parent[find(parent, x)] = find(parent, y);
16 }
17
18 void prims(int g[5][5], int V) {
19     int key[5], parent[5], used[5];
20     for(int i = 0; i < V; i++) key[i] = INT_MAX, used[i] = 0;
21     key[0] = 0;
22     parent[0] = -1;
23
24     for(int i = 0; i < V - 1; i++) {
25         int min = INT_MAX, u = -1;
26         for(int j = 0; j < V; j++)
27             if(!used[j] && key[j] < min)
28                 min = key[j], u = j;
29
30         if (u == -1) break; // Safety check
31         used[u] = 1;
```

Output

Prim:
0-1 2
1-2 3
0-3 6
1-4 5
Total 16

Kruskal:
0-1 2
1-2 3
1-4 5
0-3 6
Total 16

=== Code Execution Successful ===

Problem-9a: Dijkstra's Algorithm

Code:

```
#include <stdio.h>

#include <limits.h>

int minDist(int dist[], int used[], int V)
{
    int min = INT_MAX, idx;
    for(int i = 0; i < V; i++)
        if(!used[i] && dist[i] < min)
            min = dist[i], idx = i;
    return idx;
}

void dijkstra(int g[5][5], int src, int V)
{
    int dist[5], used[5];
    for(int i = 0; i < V; i++)
        dist[i] = INT_MAX, used[i] = 0;
    dist[src] = 0;
    for(int i = 0; i < V - 1; i++)
    {
        int u = minDist(dist, used, V);
        used[u] = 1;
        for(int v = 0; v < V; v++)
            if(g[u][v] && !used[v] && dist[u] != INT_MAX && dist[u] + g[u][v] <
dist[v])
                dist[v] = dist[u] + g[u][v];
    }
}
```

```

    }

    printf("Vertex Distance\n");

    for(int i = 0; i < V; i++)

        printf("%d %d\n", i, dist[i]);

}

int main()

{

    int g[5][5] = {

        {0,4,0,0,8},

        {4,0,8,0,11},

        {0,8,0,7,0},

        {0,0,7,0,9},

        {8,11,0,9,0}

    };

    dijkstra(g, 0, 5);

    return 0;

}

```

```

main.c
1 #include <stdio.h>
2 #include <limits.h>
3
4 int minDist(int dist[], int used[], int V) {
5     int min = INT_MAX, idx = -1;
6     for(int i = 0; i < V; i++)
7         if(!used[i] && dist[i] < min)
8             min = dist[i], idx = i;
9     return idx;
10 }
11
12 void dijkstra(int g[5][5], int src, int V) {
13     int dist[5], used[5];
14     for(int i = 0; i < V; i++)
15         dist[i] = INT_MAX, used[i] = 0;
16     dist[src] = 0;
17
18     for(int i = 0; i < V - 1; i++) {
19         int u = minDist(dist, used, V);
20         if (u == -1) break; // Safety check
21         used[u] = 1;
22
23         for(int v = 0; v < V; v++)
24             if(g[u][v] && !used[v] && dist[u] != INT_MAX &&
25                 dist[u] + g[u][v] < dist[v])
26                 dist[v] = dist[u] + g[u][v];
27     }
28
29     printf("Vertex Distance\n");
30     for(int i = 0; i < V; i++)
31         printf("%d %d\n", i, dist[i]);
32 }

```

Vertex Distance

```

0 0
1 4
2 12
3 17
4 8

```

=== Code Execution Successful ===

Problem-9b: Bellman-Ford Algorithm

Code:

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
typedef struct {
```

```
    int u, v, w;
```

```
} Edge;
```

```
void bellman_ford(int V, int E, Edge *edges, int src)
```



```

{
    int dist[10];
    for(int i = 0; i < V; i++)
        dist[i] = INT_MAX / 2;
    dist[src] = 0;
    for(int i = 1; i <= V - 1; i++)
        for(int j = 0; j < E; j++)
            if(dist[edges[j].u] + edges[j].w < dist[edges[j].v])
                dist[edges[j].v] = dist[edges[j].u] + edges[j].w;
    for(int j = 0; j < E; j++)
        if(dist[edges[j].u] + edges[j].w < dist[edges[j].v])
        {
            printf("Graph contains a negative-weight cycle\n");
            return;
        }
    printf("Vertex Distance\n");
    for(int i = 0; i < V; i++)
        printf("%d %d\n", i, dist[i]);
}

int main()
{
    Edge edges[] = {
        {0,1,6},{0,2,7},{1,2,8},{1,3,5},
        {1,4,-4},{2,3,-3},{2,4,9},{3,1,-2},
        {4,0,2},{4,3,7}
    };

```

```

bellman_ford(5, 10, edges, 0);

return 0;

}

```

The screenshot shows a C++ IDE with a file named 'main.c'. The code implements the Bellman-Ford algorithm. It defines an 'Edge' struct with 'u', 'v', and 'w' fields. The 'bellman_ford' function takes the number of vertices (V), edges (E), an array of edges, and a source vertex (src). It initializes a distance array 'dist' of size 10, setting all values to INT_MAX except for the source vertex. It then iterates over all edges V-1 times, updating the distance to each vertex. After the iterations, it prints the 'Vertex Distance' for each vertex. The output shows distances for vertices 0 through 4: 0, 2, 7, 4, and -2. A message 'Code Execution Successful' is displayed in the output pane.

```

main.c
1 #include <stdio.h>
2 #include <limits.h>
3
4 typedef struct {
5     int u, v, w;
6 } Edge;
7
8 void bellman_ford(int V, int E, Edge *edges, int src) {
9     int dist[10];
10    for(int i = 0; i < V; i++)
11        dist[i] = INT_MAX;
12    dist[src] = 0;
13
14    for(int i = 1; i <= V - 1; i++)
15        for(int j = 0; j < E; j++)
16            if(dist[edges[j].u] != INT_MAX &&
17               dist[edges[j].u] + edges[j].w < dist[edges[j].v])
18                dist[edges[j].v] = dist[edges[j].u] + edges[j].w;
19
20    for(int j = 0; j < E; j++)
21        if(dist[edges[j].u] != INT_MAX &&
22           dist[edges[j].u] + edges[j].w < dist[edges[j].v]) {
23            printf("Graph contains a negative-weight cycle\n");
24            return;
25        }
26
27    printf("Vertex Distance\n");
28    for(int i = 0; i < V; i++)
29        printf("%d %d\n", i, dist[i]);
30
31 }

```

Output

```

Vertex Distance
0 0
1 2
2 7
3 4
4 -2

=== Code Execution Successful ===

```

Problem-10a: 0/1 Knapsack

Code:

```

#include <stdio.h>

int knapsack_01(int *wt, int *val, int n, int W)
{
    int dp[n+1][W+1];

    for(int i=0;i<=n;i++)
        for(int w=0;w<=W;w++)
            if(i==0 || w==0) dp[i][w]=0;
            else if(wt[i-1]<=w)
                dp[i][w] = dp[i-1][w] > dp[i-1][w-wt[i-1]] + val[i-1] ? dp[i-1][w] :
                dp[i-1][w-wt[i-1]] + val[i-1];
            else dp[i][w]=dp[i-1][w];

    return dp[n][W];
}

int main()

```

```
{  
    int wt[] = {2,3,4,5};  
    int val[] = {3,4,5,6};  
    int n = 4;  
    int W = 5;  
    printf("%d\n", knapsack_01(wt,val,n,W));
```

```

return 0;

}

```

The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a dynamic programming solution for the 0/1 knapsack problem. It defines a function 'knapsack_01' that takes weight array 'wt', value array 'val', number of items 'n', and knapsack capacity 'W'. The function uses a 2D DP table 'dp' to calculate the maximum value. In the 'main' function, it initializes 'wt' as {2,3,4,15}, 'val' as {3,14,5,6}, 'n' as 4, and 'W' as 5. It then prints the result of 'knapsack_01(wt, val, n, W)' and returns 0. The output pane on the right shows '17' and a message '=== Code Execution Successful ==='.

```

main.c
1 #include <stdio.h>
2 int knapsack_01(int *wt, int *val, int n, int W)
3 {
4     int dp[n+1][W+1]; for(int i=0;i<=n;i++)
5     for(int w=0;w<=W;w++) if(i==0 || w==0) dp[i][w]=0; else if(wt[i-1]<=w)
6     dp[i][w] = dp[i-1][w] > dp[i-1][w-wt[i-1]] + val[i-1] ? dp[i-1][w] : dp[i-1][w-wt[i-1]]
7     + val[i-1];
8     else dp[i][w]=dp[i-1][w]; return dp[n][W];
9 }
10 int main()
11 {
12     int wt[] = {2,3,4,15};
13     int val[] = {3,14,5,6}; int n = 4;
14     int W = 5;
15     printf("%d\n", knapsack_01(wt, val, n, W));
16     return 0;
17 }

```

Output

```

17
=== Code Execution Successful ===

```

Problem-10b: Minimum Number of Coins

Code:

```

#include <stdio.h>

#include <limits.h>

int min_coins(int *coins, int n, int amount)
{
    int dp[amount+1];

    for(int i=0;i<=amount;i++)
        dp[i] = INT_MAX-1;

    dp[0] = 0;

    for(int i=1;i<=amount;i++)
        for(int j=0;j<n;j++)
            if(coins[j]<=i && dp[i-coins[j]]+1 < dp[i])
                dp[i] = dp[i-coins[j]] + 1;

    return dp[amount]==INT_MAX-1 ? -1 : dp[amount];
}

int main()
{
    int coins[] = {1,5,6,9};

```

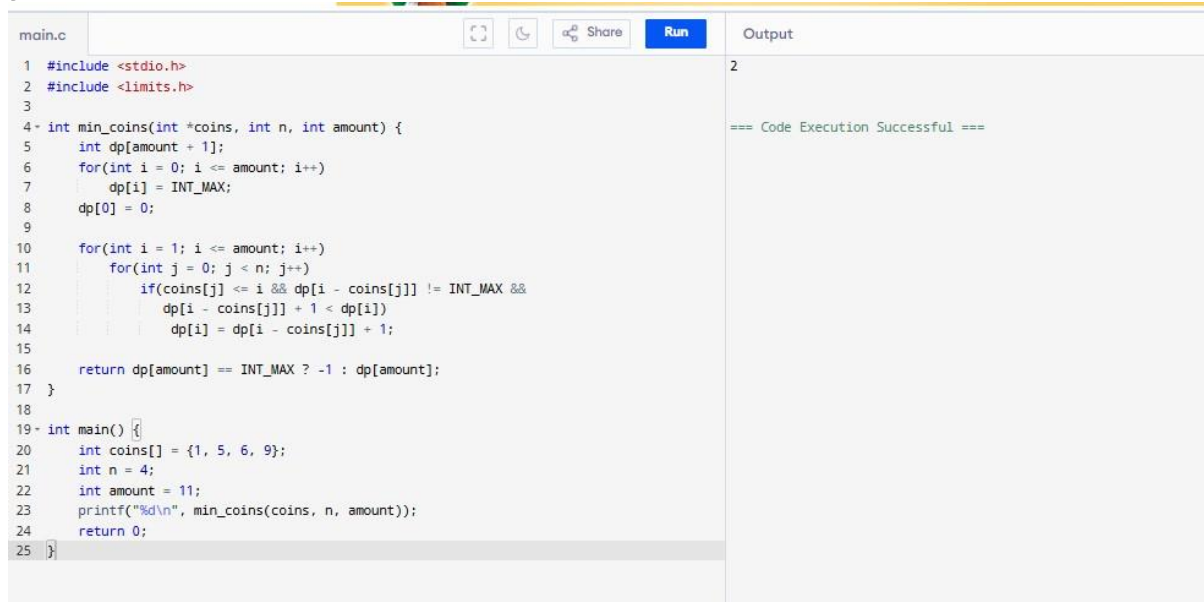
```
int n = 4;

int amount = 11;

printf("%d\n", min_coins(coins,n,amount));

return 0;

}
```



The screenshot shows a C code editor with a file named 'main.c'. The code implements a dynamic programming solution for finding the minimum number of coins to make a given amount. The function `min_coins` takes an array of coin denominations, the number of coins `n`, and the target amount. It uses a DP array `dp` to store the minimum number of coins for each amount up to `amount`. The `main` function initializes the coin array to `{1, 5, 6, 9}`, sets `n = 4` and `amount = 11`, and prints the result of `min_coins`.

```
main.c
1 #include <stdio.h>
2 #include <limits.h>
3
4 int min_coins(int *coins, int n, int amount) {
5     int dp[amount + 1];
6     for(int i = 0; i <= amount; i++)
7         dp[i] = INT_MAX;
8     dp[0] = 0;
9
10    for(int i = 1; i <= amount; i++)
11        for(int j = 0; j < n; j++)
12            if(coins[j] <= i && dp[i - coins[j]] != INT_MAX &&
13                dp[i - coins[j]] + 1 < dp[i])
14                dp[i] = dp[i - coins[j]] + 1;
15
16    return dp[amount] == INT_MAX ? -1 : dp[amount];
17 }
18
19 int main() {
20     int coins[] = {1, 5, 6, 9};
21     int n = 4;
22     int amount = 11;
23     printf("%d\n", min_coins(coins, n, amount));
24     return 0;
25 }
```

Output

```
2
=== Code Execution Successful ===
```

Problem-10c: Minimum Ways of Coin Change (Count of Combinations)

Code:

```
#include <stdio.h>
```

```
int count_ways(int *coins, int n, int amount) {
```

```
    int dp[amount + 1];
```

```
    for(int i = 0; i <= amount; i++)
```

```
        dp[i] = 0;
```

```
    dp[0] = 1;
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = coins[i]; j <= amount; j++)
```

```
            dp[j] += dp[j - coins[i]];
```

```
    return dp[amount];
```

```
}
```

```
int main() {
```

```
    int coins[] = {1, 2, 3};
```

```
    int n = 3;
```

```
    int amount = 4;
```

```
    printf("%d\n", count_ways(coins, n, amount));
```

```
    return 0;
```

```
}
```

```
main.c
1 #include <stdio.h>
2
3 int count_ways(int *coins, int n, int amount) {
4     int dp[amount + 1];
5     for(int i = 0; i <= amount; i++)
6         dp[i] = 0;
7     dp[0] = 1;
8
9     for(int i = 0; i < n; i++)
10         for(int j = coins[i]; j <= amount; j++)
11             dp[j] += dp[j - coins[i]];
12
13     return dp[amount];
14 }
15
16 int main() {
17     int coins[] = {1, 2, 3};
18     int n = 3;
19     int amount = 4;
20     printf("%d\n", count_ways(coins, n, amount));
21     return 0;
22 }
```

Output

4

=== Code Execution Successful ===

Problem-10d: Longest Common Subsequence (LCS)

Code:

```
#include <stdio.h>

#include <string.h>

int max(int a,int b){ return a>b?a:b; }

int lcs(char *s1, char *s2, int m, int n)
{
    int dp[m+1][n+1];

    for(int i=0;i<=m;i++)
        for(int j=0;j<=n;j++)
            if(i==0 || j==0) dp[i][j]=0;
            else if(s1[i-1]==s2[j-1]) dp[i][j]=dp[i-1][j-1]+1;
            else dp[i][j]=max(dp[i-1][j], dp[i][j-1]);

    return dp[m][n];
}

int main()
{
    char s1[]="ABCBBDAB";
    char s2[]="BDCAB";

    printf("%d\n", lcs(s1,s2,strlen(s1),strlen(s2)));

    return 0;
}
```

}

main.c		Share	Run	Output
1	#include <stdio.h>			4
2				
3	int lis(int *arr, int n) {			
4	int dp[n];			=== Code Execution Successful ===
5	int maxlen = 1;			
6				
7	for(int i = 0; i < n; i++)			
8	dp[i] = 1;			
9				
10	for(int i = 1; i < n; i++)			
11	for(int j = 0; j < i; j++)			
12	if(arr[i] > arr[j] && dp[i] < dp[j] + 1)			
13	dp[i] = dp[j] + 1;			
14				
15	for(int i = 0; i < n; i++)			
16	if(dp[i] > maxlen)			
17	maxlen = dp[i];			
18				
19	return maxlen;			
20	}			
21				
22	int main() {			
23	int arr[] = {10, 9, 2, 5, 3, 7, 101, 18};			
24	int n = 8;			
25	printf("%d\n", lis(arr, n));			
26	return 0;			
27	}			

Problem-10e: Longest Increasing Subsequence (LIS)

Code:

```
#include <stdio.h>

int lis(int *arr, int n) {
    int dp[n];
    int maxlen = 1;

    for(int i = 0; i < n; i++)
        dp[i] = 1;

    for(int i = 1; i < n; i++)
        for(int j = 0; j < i; j++)
            if(arr[i] > arr[j] && dp[i] < dp[j] + 1)
                dp[i] = dp[j] + 1;

    for(int i = 0; i < n; i++)
        if(dp[i] > maxlen)
            maxlen = dp[i];

    return maxlen;
}

int main() {
    int arr[] = {10, 9, 2, 5, 3, 7, 101, 18};
    int n = 8;
    printf("%d\n", lis(arr, n));
    return 0;
}
```